

# WSNs & IoT – Wireless Sensor Networks & Internet of Things

---

Tema 5 – Diseño de Redes

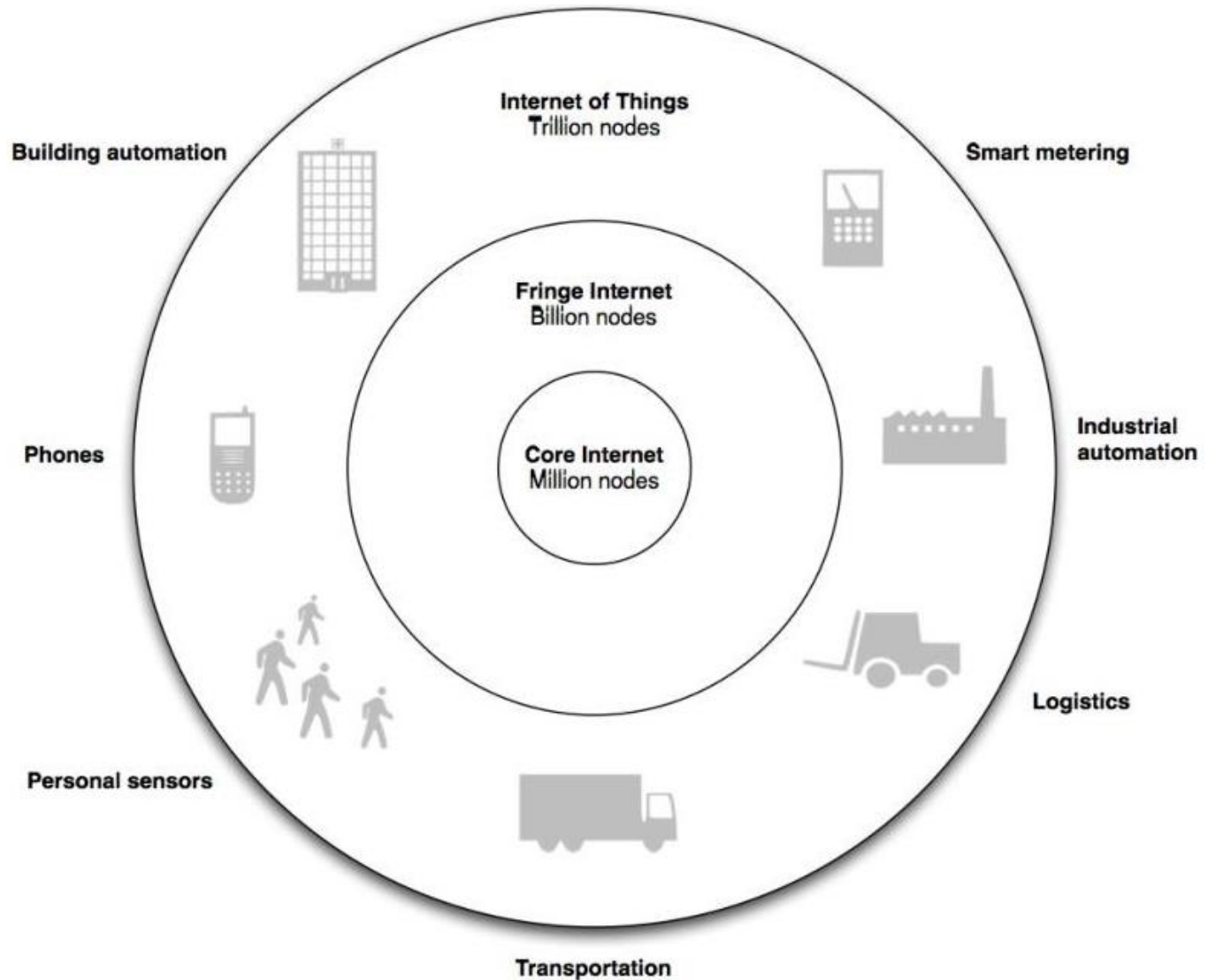
# WSNs & IoT

---

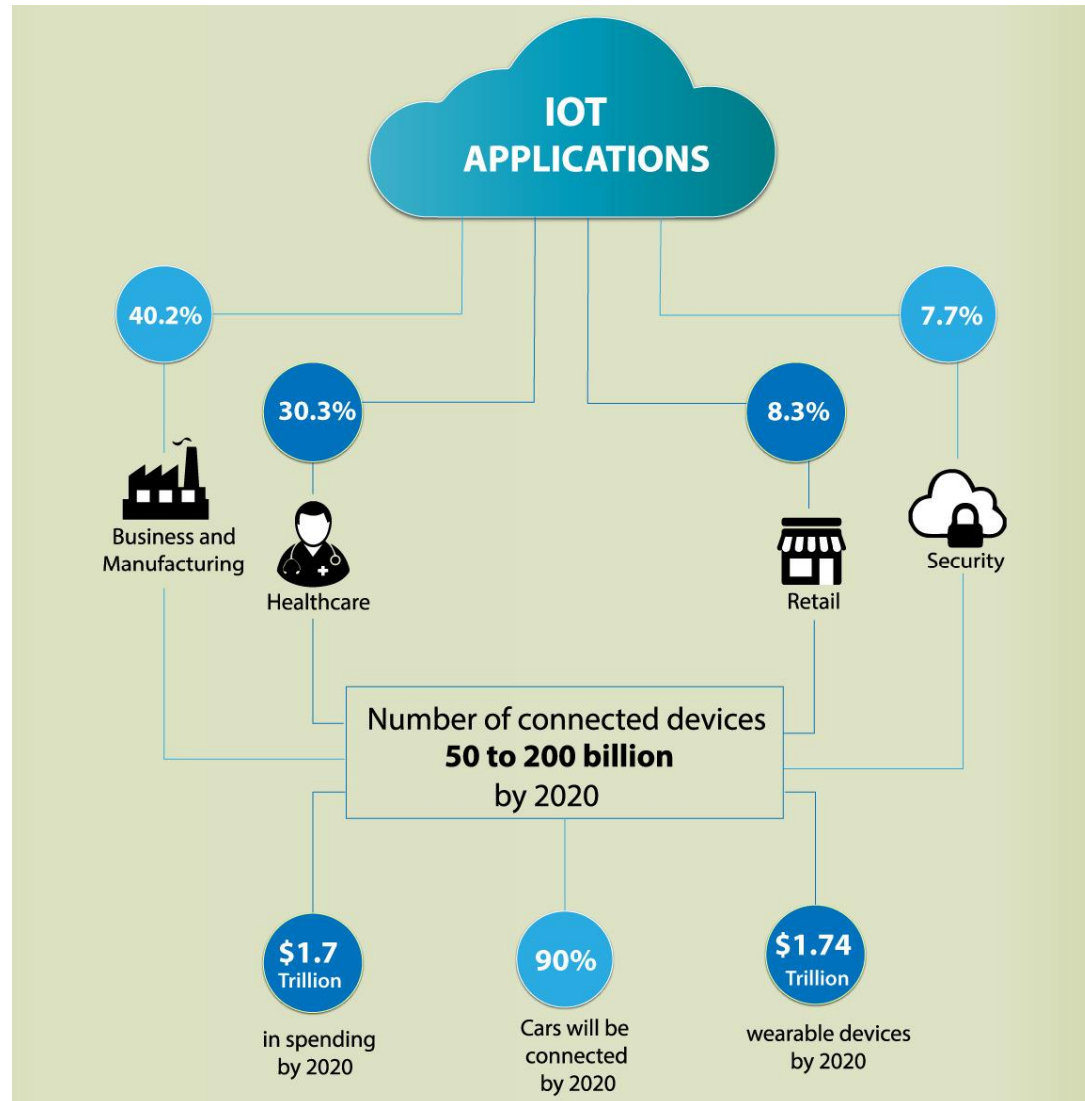
- **Protocolos de aplicación**
  - MQTT (publicación – suscripción)
  - CoAP (petición – respuesta)
- **Protocolos de red**
  - MAC: IEEE 802.15.4
  - Red: 6LoWPAN

# IoT (Internet of Things)

---



# IoT



# Protocolos de aplicación

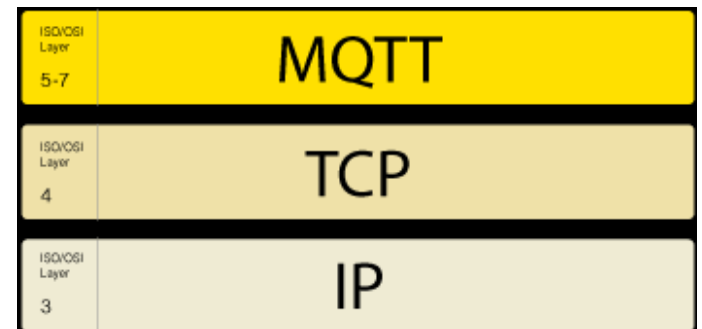
---

## Tema 5 WSNs & IoT

# MQTT

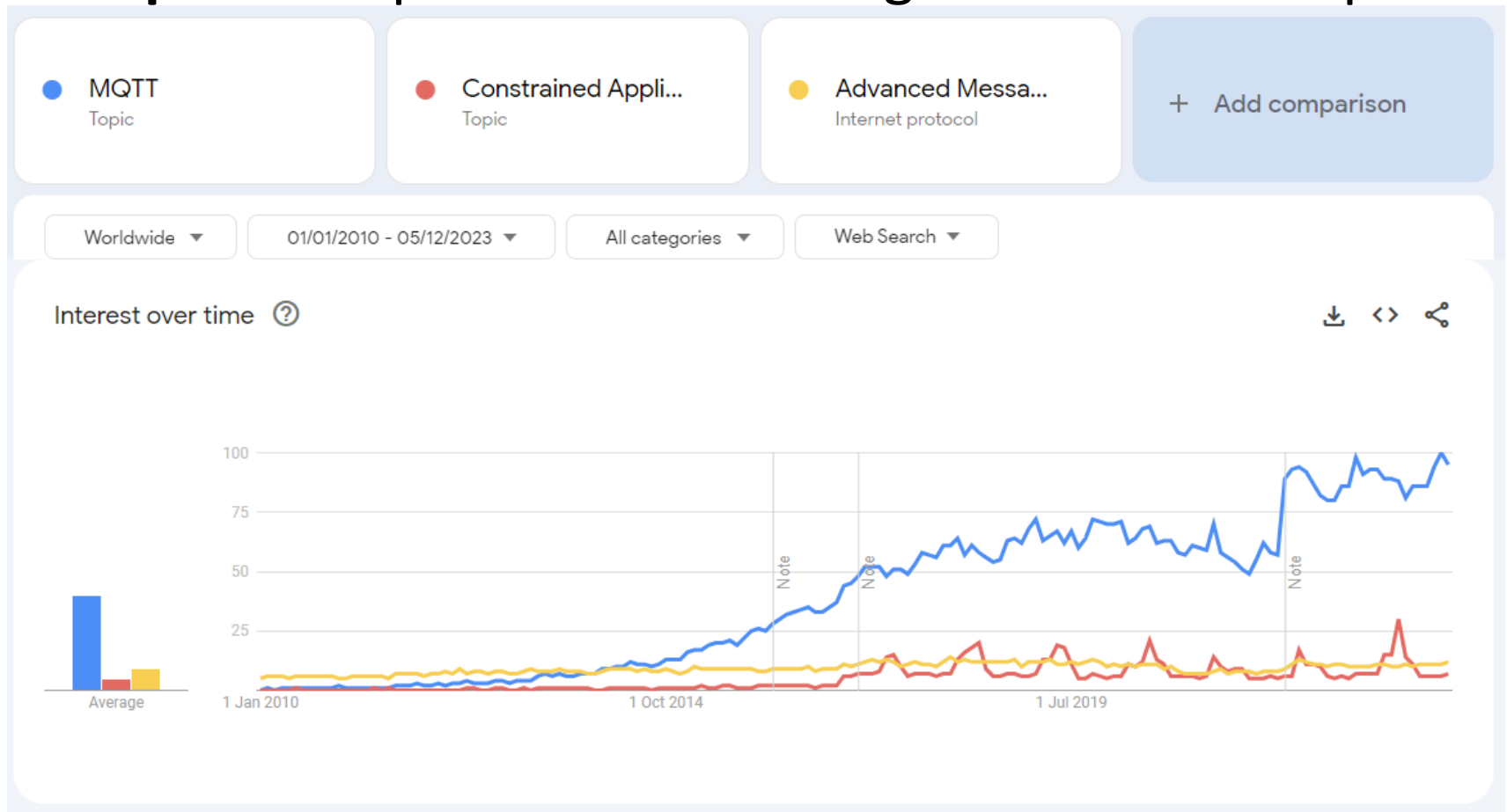
---

- Message Queuing Telemetry Transport
- **Protocolo ligero de Publicación – Suscripción**
- **Estándar** abierto **OASIS**
- Basado en **TCP**
- **Arquitectura cliente/servidor**
  - Cliente: un sensor que “publica” la info en el servidor
  - Servidor (bróker): recibe la info y la envía a los suscriptores
- **QoS**
- **Comunicación segura**
- **Persistencia**



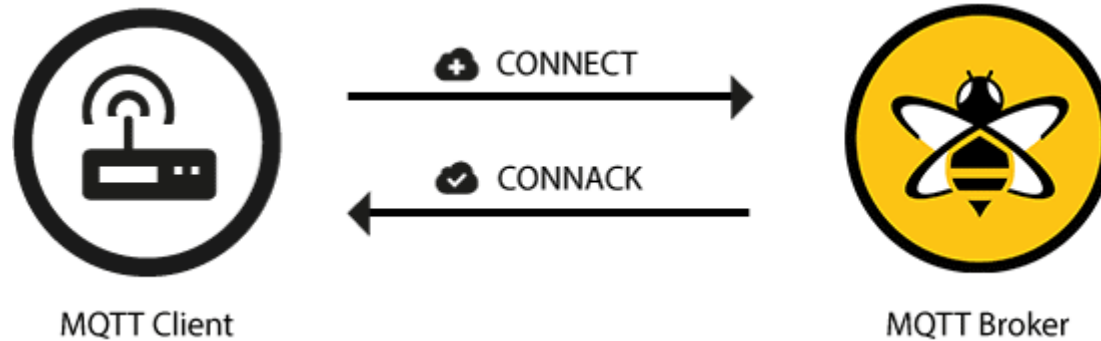
# MQTT (II)

- **MQTT-SN**: MQTT for **Sensor Networks**
- **Mosquitto**: implementación código abierto de Eclipse



# Mensajes MQTT – Conexión

- Conexión

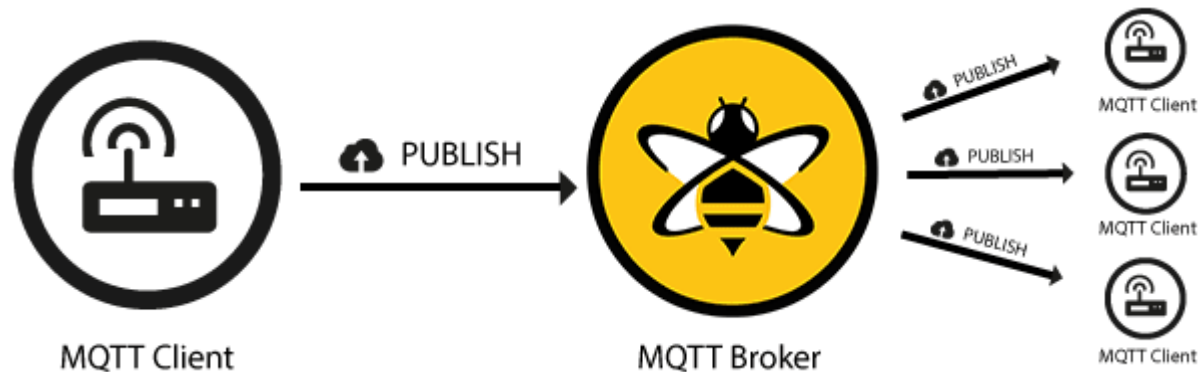


| MQTT-Packet: <b>CONNECT</b>  |                   |
|---------------------------------------------------------------------------------------------------------------|-------------------|
| contains:                                                                                                     | Example           |
| <b>clientId</b>                                                                                               | "client-1"        |
| <b>cleanSession</b>                                                                                           | true              |
| <b>username</b> (optional)                                                                                    | "hans"            |
| <b>password</b> (optional)                                                                                    | "letmein"         |
| <b>lastWillTopic</b> (optional)                                                                               | "/hans/will"      |
| <b>lastWillQos</b> (optional)                                                                                 | 2                 |
| <b>lastWillMessage</b> (optional)                                                                             | "unexpected exit" |
| <b>lastWillRetain</b> (optional)                                                                              | false             |
| <b>keepAlive</b>                                                                                              | 60                |

| MQTT-Packet: <b>CONNACK</b>  |         |
|-----------------------------------------------------------------------------------------------------------------|---------|
| contains:                                                                                                       | Example |
| <b>sessionPresent</b>                                                                                           | true    |
| <b>returnCode</b>                                                                                               | 0       |

# MQTT Messages – Publish

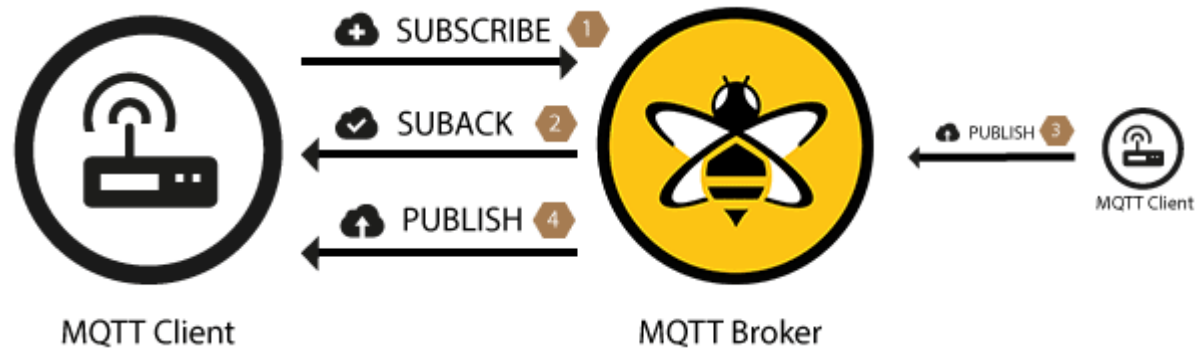
- Publicación



| MQTT-Packet:                                                                        |                    |
|-------------------------------------------------------------------------------------|--------------------|
| PUBLISH                                                                             |                    |
|  |                    |
| contains:                                                                           | Example            |
| <b>packetId</b> (always 0 for qos 0)                                                | 4314               |
| <b>topicName</b>                                                                    | "topic/1"          |
| <b>qos</b>                                                                          | 1                  |
| <b>retainFlag</b>                                                                   | false              |
| <b>payload</b>                                                                      | "temperature:32.5" |
| <b>dupFlag</b>                                                                      | false              |

# MQTT Messages – Subscribe

- Suscripción



MQTT-Packet:

## SUBSCRIBE



contains:

Example

```
packetID
topic1
topic2
...
```

MQTT-Packet:

## SUBACK



contains:

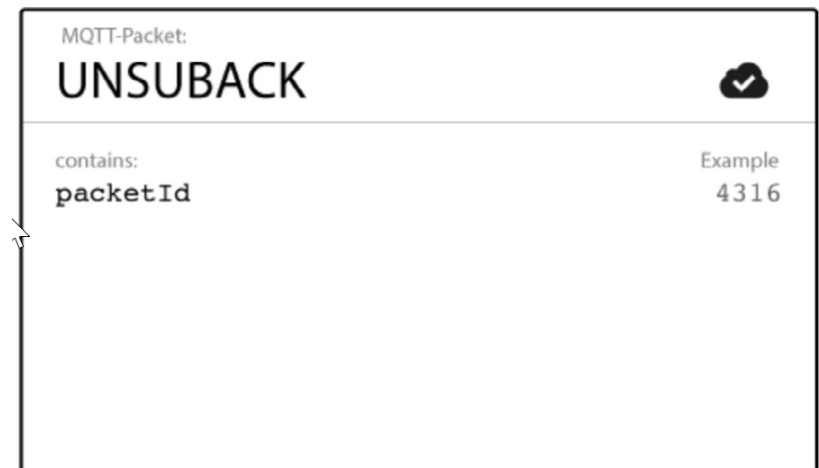
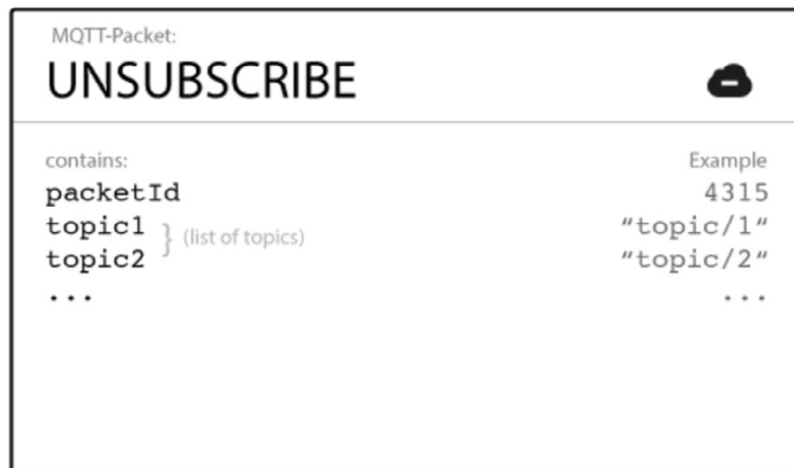
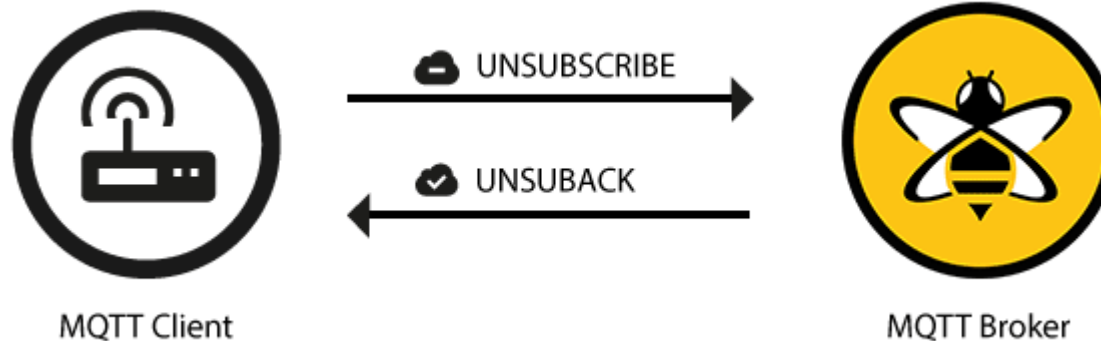
Example

```
packetId
returnCode 1 ( one returnCode for each
returnCode 2 topic from SUBSCRIBE,
...          in the same order )
...
```

```
4313
2
0
...
```

# MQTT Messages – Unsubscribe

- Desuscripción



# Conceptos básicos de MQTT

---

- **Tópicos (temas)**

- Similares a rutas. Ex.: *topic1/subtopic11/1/subtopic3*
- Comodines
  - Nivel simple '+'
  - Multinivel '#'
- Ejemplos
  - *topic1/#*
  - *topic1/+/1/#*
- Prácticas recomendadas
  - Nunca usar una '/' inicial ni espacios
  - Breves y concisos

- **Mensaje & tópico Last Will**

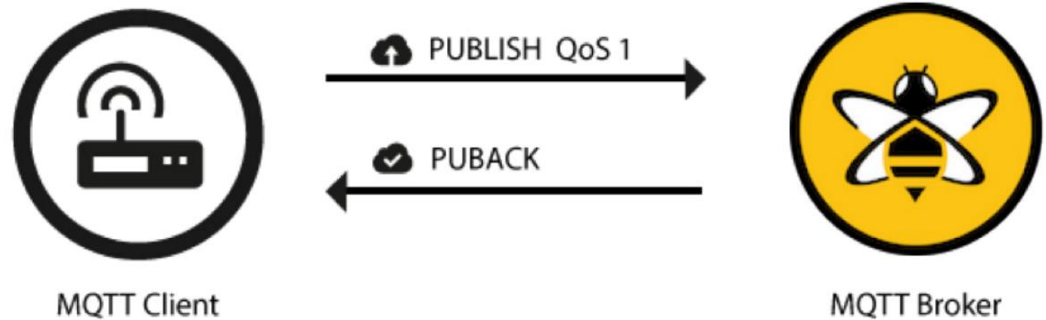
- Se establecen cuando el cliente se conecta
- El servidor publica el mensaje Last Will en el tópico Last Will cuando el cliente se desconecta de forma anormal

# MQTT QoS

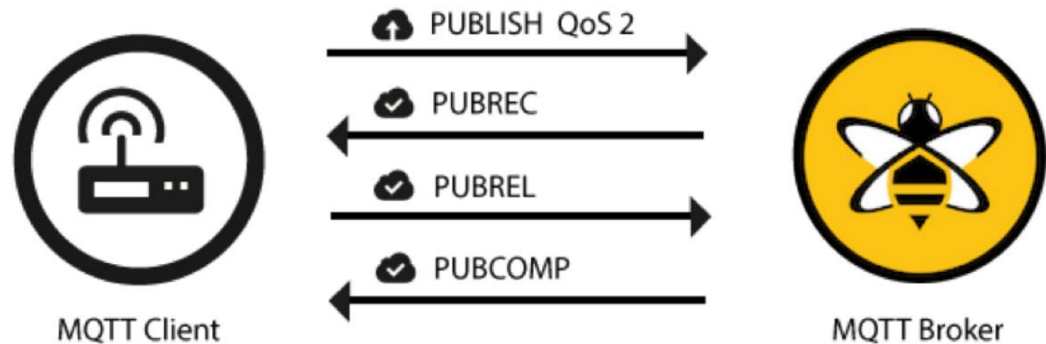
- QoS 0: *at most once*



- QoS 1: *at least once*



- QoS 2: *exactly once*



# MQTT – Taller

---

- **Instala *mosquitto***
  - `sudo apt install mosquitto mosquitto-clients`
- **Crea la PKI necesaria (certificados y claves) para soportar TLS en *mosquitto***
  - Lanza terminal como *root*
    - `sudo bash`
  - Genera un certificado y una clave CA
    - `cd /etc/mosquitto/ca_certificates`
    - `openssl req -new -x509 -days 365 -extensions v3_ca -keyout ca.key -out ca.crt`
  - Genera una clave de servidor sin cifrado
    - `cd /etc/mosquitto/certs`
    - `openssl genrsa -out server.key 2048`
  - Genera una solicitud de firma de certificado (CSR) para enviarla a la CA
    - `openssl req -out server.csr -key server.key -new`
  - Envía la CSR a la CA o fírmela con la clave de CA
    - `openssl x509 -req -in server.csr -CA ../ca_certificates/ca.crt -CAkey ../ca_certificates/ca.key -CAcreateserial -out server.crt -days 365`
  - Cambia el propietario y el grupo de *server.key* al usuario *mosquitto*
    - `chown mosquitto:mosquitto server.key`

# MQTT – Taller (II)

---

- Añade las siguientes líneas a */etc/mosquitto/mosquitto.conf*

```
log_type all

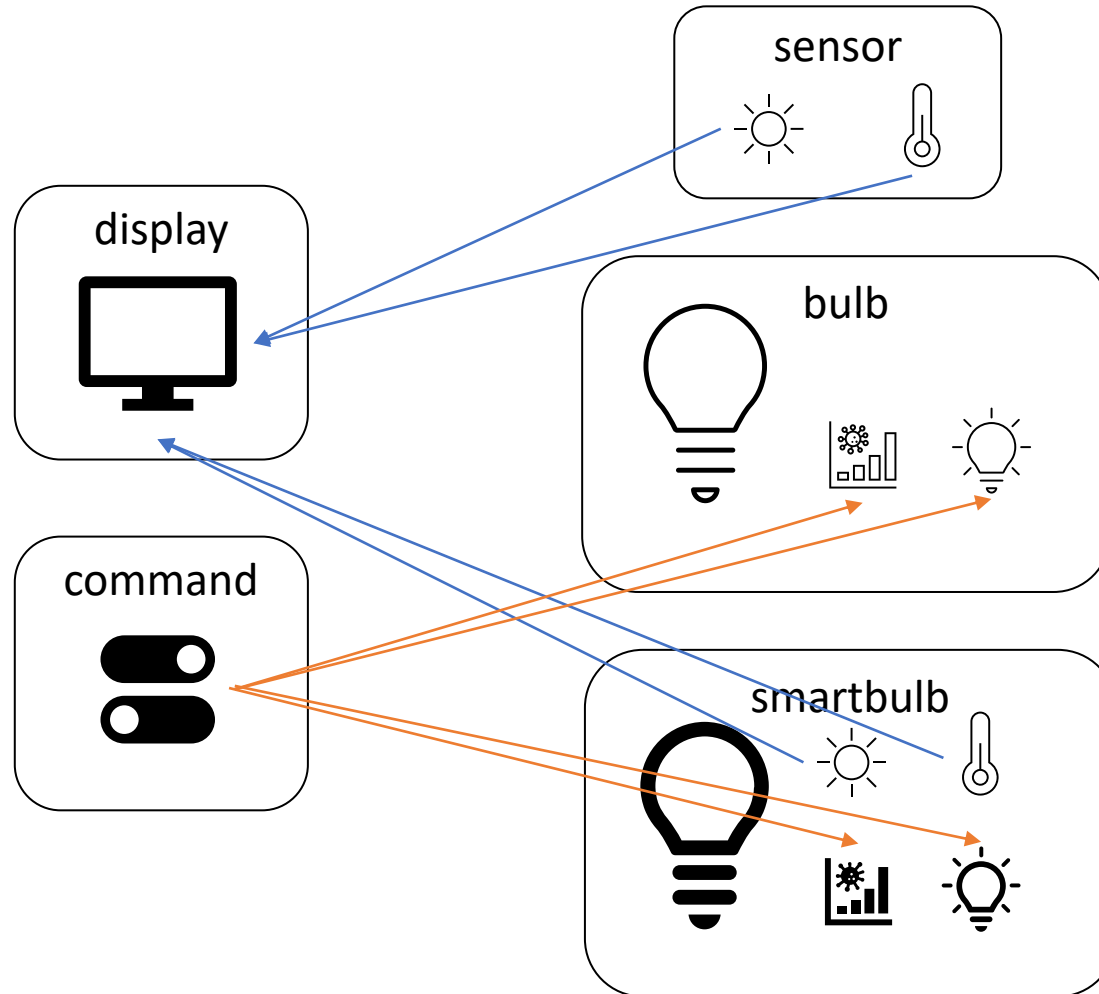
listener 1883
allow_anonymous true

listener 8883
cafile /etc/mosquitto/ca_certificates/ca.crt
certfile /etc/mosquitto/certs/server.crt
keyfile /etc/mosquitto/certs/server.key
```

- Reinicia el bróker
  - `sudo systemctl restart mosquitto`
- Muestra en una terminal aparte el *log* de *mosquitto*
  - `sudo tail -n 50 -f /var/log/mosquitto/mosquitto.log`
- Descarga el paquete .deb de MQTTX e instálalo con *dpkg -i <pkg\_name>*
- Inicia MQTTX
  - `mqttx &`

# MQTT – Taller (III)

- Escenario

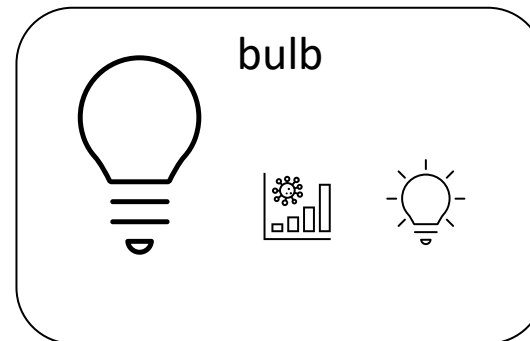
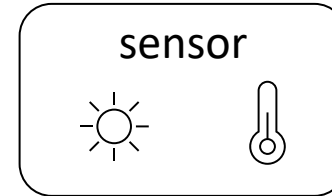
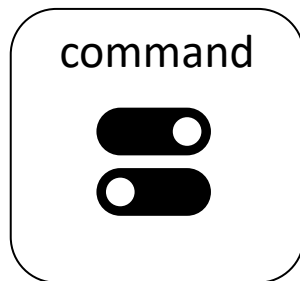
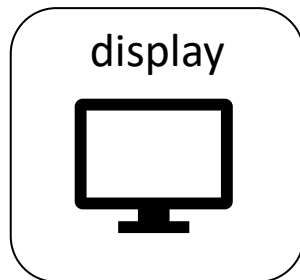


# MQTT – Taller (IV)

- Tópicos de suscripción

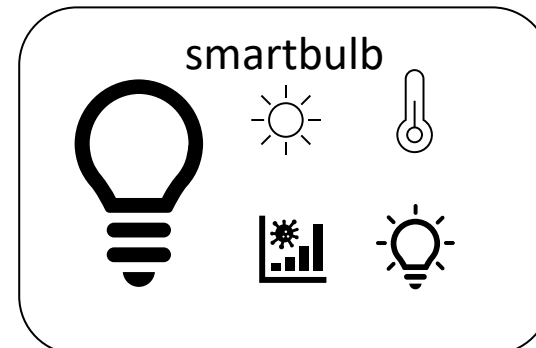
brightness/# (QoS 0)

temperature/# (QoS 1)



on/1 (QoS 0)

intensity/1 (QoS 0)



on/2 (QoS 1)

intensity/2 (QoS 2)

# MQTT – Taller (V)

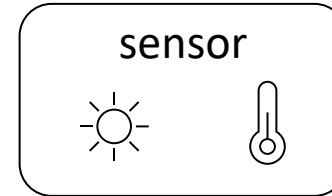
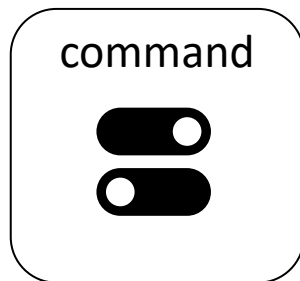
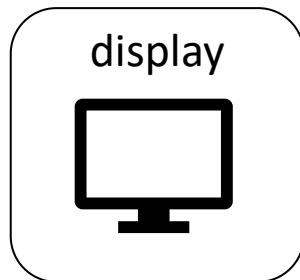
---

- En MQTTX, crea los siguientes clientes (conexiones) con el bróker con las suscripciones de tópicos correspondientes. Utiliza estos nombres para los campos Name y Client ID.
  - *sensor*
  - *bulb: on/1* (QoS 0), *intensity/1* (QoS 0)
  - *smartbulb* (TLS): *on/2* (QoS 1), *intensity/2* (QoS 2)
  - *display: brightness/#* (QoS 0), *temperature/#* (QoS 1)
  - *command* (TLS)
  - *broker-stats: \$SYS/broker/#* (QoS 0) (Retained)
- Observa cómo el bróker publica sus estadísticas en los temas *\$SYS/broker/#*
- Sal de MQTTX para cerrar todas las conexiones

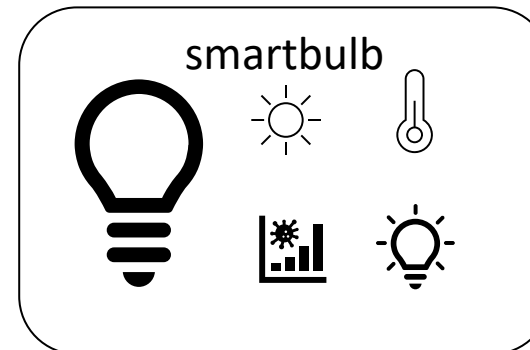
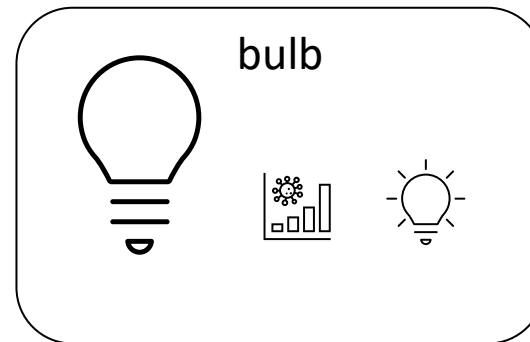
# MQTT – Taller (VI)

- Tópicos y datos de publicación

on/1: 1 (QoS 0)  
intensity/1: 3 (QoS 0)  
on/2: 0 (QoS 1)  
intensity/2: 2 (QoS 2)



brightness/1: 7 (QoS 0)  
temperature/1: 25 (QoS 1)



brightness/2: 2 (QoS 1)  
temperature/2: 20 (QoS 2)

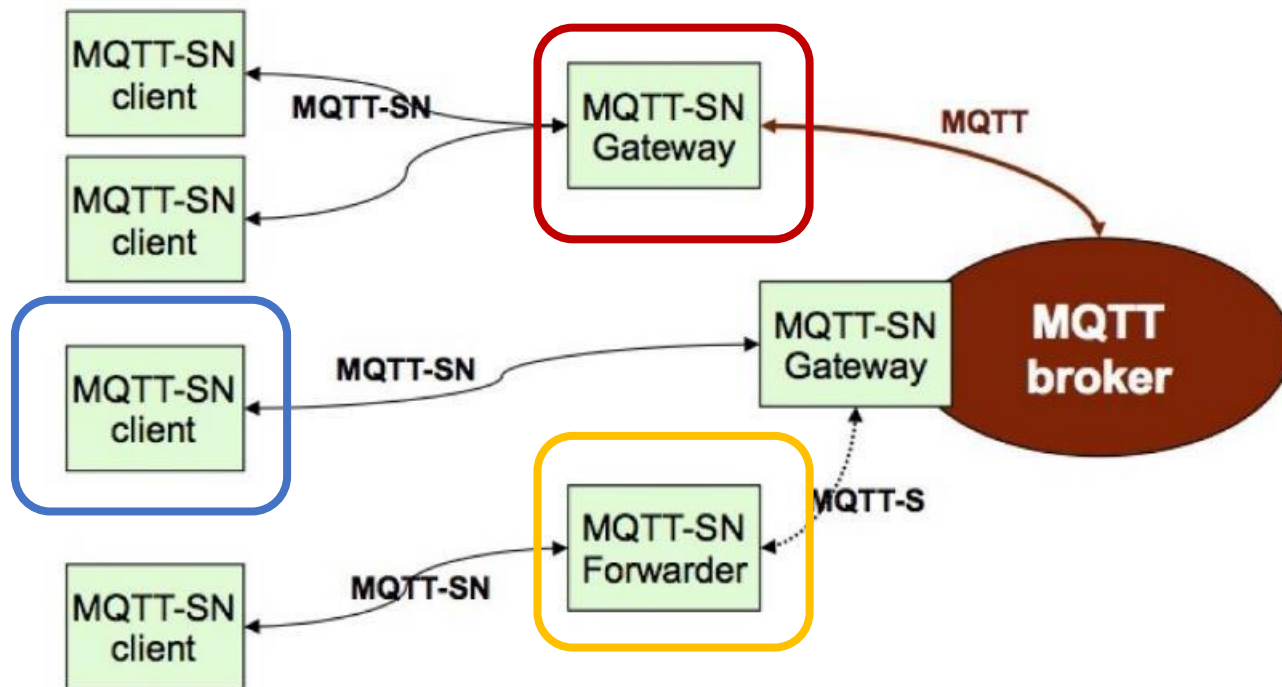
# MQTT – Taller (VII)

---

- Inicia Wireshark (como *root*) y comienza a capturar en la interfaz *localhost*
- Inicia MQTTX y conecta todos los clientes (excepto *broker-stats*)
- Examina el *log* de *mosquitto* y la captura de Wireshark
- Aplica el filtro *mqtt* en Wireshark para visualizar solo los paquetes MQTT
- Quita el filtro *mqtt* para ver los paquetes MQTT transmitidos a través de TLS
- Publica los siguientes datos y analiza los intercambios de mensajes. Explica cómo se determina la QoS en cada caso
  - *sensor*
    - *brightness/1: 7* (QoS 0)
    - *temperature/1: 25* (QoS 1)
  - *smartbulb*
    - *brightness/2: 2* (QoS 1)
    - *temperature/2: 20* (QoS 2)
  - *command*
    - *on/1: 1* (QoS 1),
    - *on/2: 0* (QoS 2),
    - *intensity/2* (QoS 1),
    - *intensity/2* (QoS 2)

# MQTT-SN

- MQTT for **Sensor Networks**
- **Mejoras**
  - **UDP** en lugar de TCP
  - **Reducción** de la **carga útil**



# MQTT-SN – Arquitectura & Mejoras

---

- **Arquitectura**

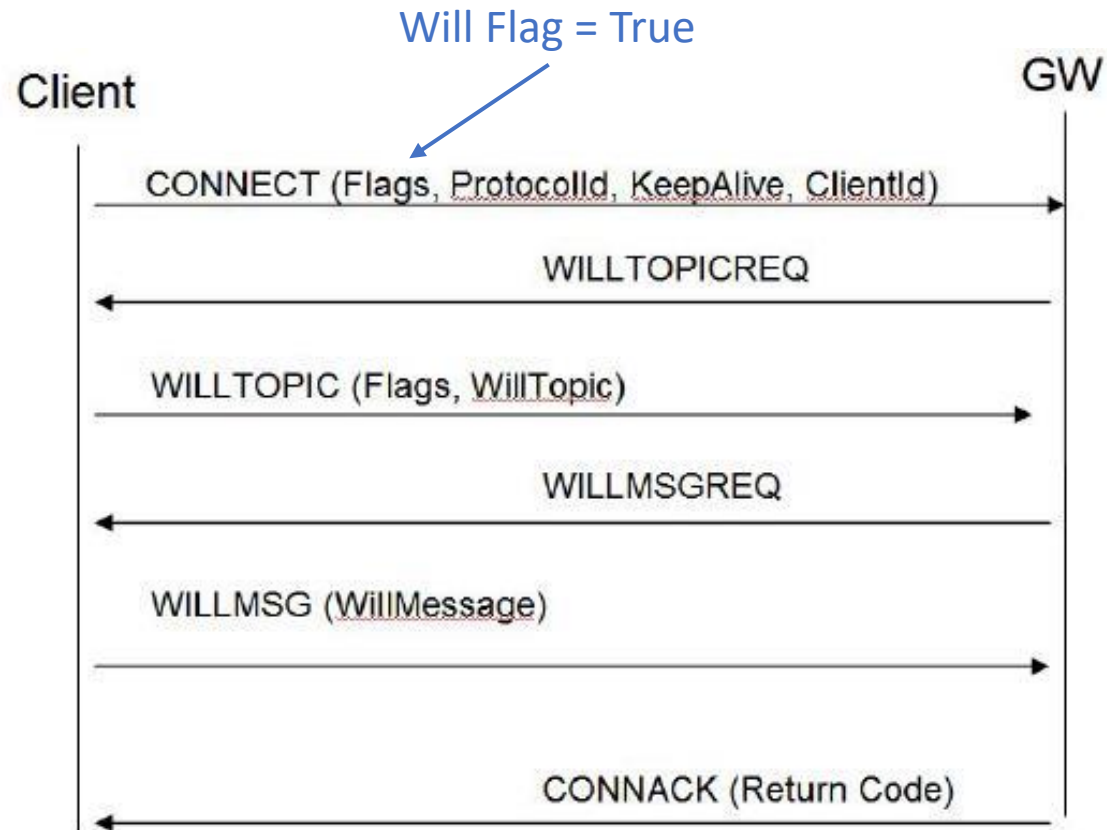
- *Gateway*: MQTT  $\leftrightarrow$  MQTT-SN
- *Forwarder*: MQTT-SN
- Cliente

- **Mejoras**

- Mensaje **CONNECT** dividido en 3 mensajes
  - Uno obligatorio
  - Dos opcionales: mensaje y tópico de última voluntad
- **Registro de tópicos**: evita nombres largos de tópicos en mensajes
- **Tópicos fijos cortos** (2B)
- **Identificadores de tópico predefinidos**, no es necesario registrarlos
- Procedimiento de **descubrimiento de gateway**
- Soporte **persistencia del mensaje y tópico de última voluntad**
- Soporte **cliente dormido**: mensajes se almacenan en búfer en la puerta de enlace

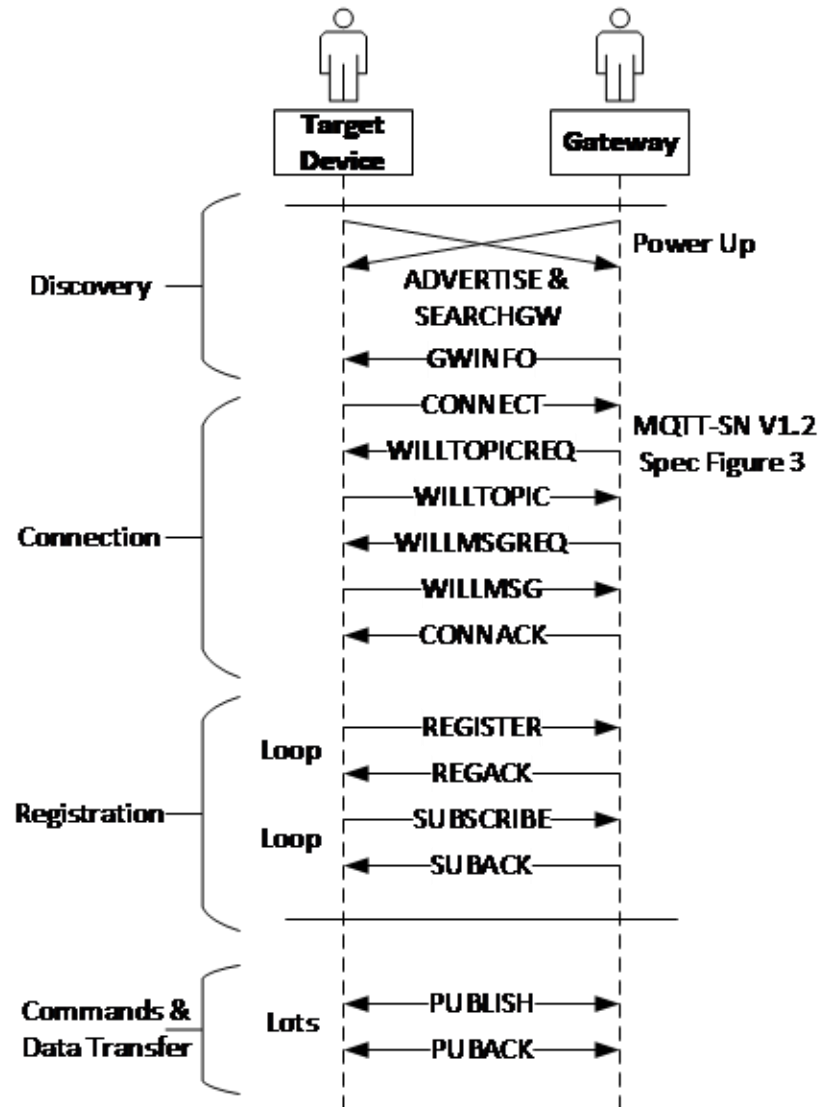
# MQTT-SN Connect

---



Si Will flag = False, se devuelve un mensaje CONNACK

# MQTT-SN – Resumen mensajes



# MQTT-SN – Taller

---

- Descarga y compila RSMB (*Really Simple Message Broker*)
  - <https://github.com/eclipse/mosquitto.rsmb>
  - `cd rsmb/src`
  - `make`
- Descarga y compila *mqtt-sn-tools*
  - <https://github.com/njh/mqtt-sn-tools>
  - `make`
- Crea el fichero *broker.cfg*  
`trace_output protocol`

```
# normal MQTT listener
listener 1883 INADDR_ANY
# MQTT-SN listener
listener 1884 INADDR_ANY mqttts
```

# MQTT-SN Taller (II)

---

- Lanza el RSMB
  - `./broker_mqtts`
- Lanza *Wireshark* (como *root*) y empieza a capturar en la interfaz *lo*
  - *Edit -> Preferences -> Protocols -> MQTT-SN -> Port: 1884*
- Lanza un cliente que se suscriba al tópico 'mytopic' QoS 1
  - `./mqtt-sn-sub -V -t mytopic -p 1884 -k 60 -i sub1 -q 1`
- Analiza el intercambio de mensajes en el *log* del RSMB y en la captura de *Wireshark*
  - Conexión
  - Registro de tópico
- Lanza un cliente que publique en el tópico 'mytopic' QoS 1
  - `./mqtt-sn-pub -k 60 -p 1884 -t mytopic -m Hello -i pub1 -q 1`
- Analiza el intercambio de mensajes